INT161 · Basic Backend Development

Template and CRUD Practice

Save the project as a template, copy the three layers for a new resource, test them one at a time, and read your first errors.

Project Template | A New Resource in Three Layers | Test One Layer at a Time

Reading Your First Errors | Lab: Actor CRUD | Glossary

01

Project Template

The lecturer planned to teach **exceptions** today, but spent the class on this: a **project template**, then a new **resource** built from it. Errors came up along the way, and the full topic moves to the next class.

- The template already has the folders: configs (the database settings), repositories, **services** and **routes**, each with an example file, plus app.js.
- The example **repository** keeps its SQL in constants, one per operation, and uses a variable for the table name. To reuse it you change the table name and the **entity name**. That is the idea.
- Do not edit the template project itself. Save it, then work in projects made from it.

1. Unzip the template and check the folder is not nested twice.
2. Put the folder where your IDE keeps projects, then open it. Choose not to trust or modify anything you do not need.
3. Run **npm** install in the project. The node_modules folder is rebuilt, so it should never be copied around.
4. WebStorm: Tools, Save Project as Template, give it a name.
5. From then on, New Project lists your template. Pick it, name the project, and the folders and code are copied in for you.

after creating a project from the template

```
$ npm install
# then set the database name in configs/db-config.js
```

> **Using AI**
>
> Not sure how to save a template in VS Code? Ask an AI how. Use it to build your knowledge, not to do the work for you. Companies do not always pay for tokens for everyone, so you need to be able to work without it.

02

A New Resource in Three Layers

The demo database was Sakila and the new resource was country. Same recipe for any table.

1. Repository

Copy the example repository, rename it country-repository, change the table and entity name.

2. Service

Copy the example service, rename it, and point it at the country repository.

3. Route

Copy the example route, rename it, and point it at the country service.

4. App

Register the **router** under its path in app.js.

repositories/country-repository.js

```
import db from '../configs/db-config.js';
const TABLE_NAME = 'country';
const ENTITY_NAME = 'Country';
const FIND_ALL_SQL = `SELECT * FROM ${TABLE_NAME}`;
const FIND_ONE_SQL = `SELECT * FROM ${TABLE_NAME} WHERE country_id = ?`;
const CREATE_SQL = `INSERT INTO ${TABLE_NAME} SET ?`;
const UPDATE_SQL = `UPDATE ${TABLE_NAME} SET ? WHERE country_id = ?`;
const REMOVE_SQL = `DELETE FROM ${TABLE_NAME} WHERE country_id = ?`;
export async function findAll() {
  const [rows] = await db.query(FIND_ALL_SQL);
  return rows;
}
export async function findOne(id) {
  const [rows] = await db.query(FIND_ONE_SQL, [id]);
  if (rows.length === 0) throw new Error(`${ENTITY_NAME} with id ${id} was not found`);
  return rows[0];
}
```

- SET ? takes a whole object. The field names of the object must match the column names. mysql2 builds the SET list for you.
- The table has an **auto-increment** key and a last_update **timestamp** that fills itself. Send only the country name.
- After an insert, the repository reads the new row back by its insert id. That is why create returns the full row, not the raw database answer.
- Rename carefully. If a name appears many times, use find and replace (a **refactor**) instead of editing one by one. Names in the template, such as id, may not match your table.

app.js

```
import express from 'express';
import countryRouter from './routes/country-route.js';
const app = express();
app.use(express.json()); // before the router, or req.body is undefined
app.use('/countries', countryRouter);
app.listen(3000);
```

03

Test One Layer at a Time

Test each layer before you stack the next one. When you write all three and test once, you cannot tell which layer failed.

1. Repository

Import it into a small test file and call each function. Print the result.

2. Service

Same test file, but call the service. It should give the same results.

3. Route

Cannot be called from a script, because it needs a real **HTTP request**. Run the app and use **Postman**.

tests/test-country-repository.js

```
import * as repo from '../repositories/country-repository.js';
console.log(await repo.findAll());
console.log(await repo.findOne(94));
// console.log(await repo.create({ country: 'Thailand' })); // no id: the database makes it
```

- Update and delete: test with an id you created yourself. Do not touch rows other people might use.
- Always check the result of a change in the database too, not only the answer your code printed.

The route layer is the **controller**. It reads the request and answers with the standard codes.

`GET /countries`

200 with an array

`GET /countries/:id`

200 with one object

`POST /countries`

201 with the new row

`PUT /countries/:id`

200 with the changed row

`DELETE /countries/:id`

204 with no body

- The **body** is already an object after **express**.json(). Pass req.body straight on if you trust it. We do not validate yet, so we assume the **front end** sends it right.
- Delete answers 204 and sends nothing. To prove it worked, fetch the same id again and expect not found.

04

Reading Your First Errors

Your first real error appeared in class. The template said id, but the table calls its key country_id. The lecturer's rule: read the error message. It is the only reliable clue. Do not compare your code to the original and guess.

Message | First line of your code | Exit code

```
Error: Unknown column 'id' in 'where clause'
    at async findOne (src/repositories/country-repository.js:24:29)
Process exited with code 1
```

It tells you

Say what is wrong. The database does not know a column called id.

So you

Open the table and read its real column names. Here the key is country_id, so the template's id must change.

Example only. Your file name and line will differ.

- In the IDE the file in the **stack trace** is a link. Click it to jump to the line.
- Not found is an error too. Deleting id 110 twice: the second call finds nothing, so findOne **throws**. Left unhandled, **Node** exits with code 1 and the **server** dies. In production that means the app is down.

> ⚠ **Next class**
>
> Handling these errors so the server survives and answers with a proper **status code** is the topic of the next class.

05

Lab: Actor CRUD

The class task: build **CRUD** for the actor resource in Sakila. It has four columns. You manage two: first_name and last_name. The id is automatic, and last_update fills itself.

- Three layers: repository, service, route. Five **endpoints**: get all, get by id, create, update, remove.
- Hand in a zip of the project and a PDF of screenshots. One screenshot per endpoint, showing the **URL** and the data. For get all, one screen is enough. Add a short explanation of what you did.
- Delete node_modules before you zip. It is huge and the reader reinstalls it anyway. Points are deducted if you forget.
- Prove the delete: show the 204, then get the same id and show it is gone.
- Talk to friends and ask for help like colleagues do. Do not copy their code.
- Finished early? Try the city resource. It is harder because it has a relationship.

> **Also mentioned**
> Projects built with Express can also be run with Bun, which the lecturer says is faster. Try it if you like.

06

Glossary

auto-increment
: The database counts up the id for you, so you never send one on INSERT and it never duplicates.

body
: The data a request carries after the headers. For an API it is usually JSON.

controller
: The layer that receives the HTTP request and sends the response. It delegates the real work.

CRUD
: Create, Read, Update, Delete: the four basic things you do with data.

endpoints
: One method plus path an API answers, such as GET /api/subjects.

entity name
: The display name of a resource, such as Country. It can differ from the table name.

exceptions

An event that breaks the normal flow of a program. It is thrown as an object holding the details.

express

A small web framework for Node.js. It adds routing and middleware on top of the built-in http module.

front end

The part users see and touch. It runs in the browser or on the phone.

HTTP

HyperText Transfer Protocol: the plain-text language a browser and a web server use to talk. One side sends a request, the other sends a response.

Node

A runtime that runs JavaScript outside the browser, so the same language can write the back end.

npm

Node Package Manager: installs libraries such as express into node_modules.

Postman

A tool for sending any HTTP method to your API, so you can test it before the front end exists.

project template

A saved starting project. A new project made from it already has the folders and the basic code.

refactor

Change the structure or names of code without changing what it does, such as renaming a field everywhere.

repository

The layer that reads and writes the database. No business rules, no HTTP.

request

What the client sends: method, URL, headers and sometimes a body.

resource

A thing the API gives access to, such as a product or a customer. In this course it maps to a database table.

router

Matching an HTTP method and a URL path to the function that handles it.

routes

Matching an HTTP method and a URL path to the function that handles it.

server

A program (or the machine running it) that waits for requests and answers them.

services

The layer that holds the business rules. It calls repositories and knows nothing about HTTP.

stack trace

The list of function calls, with file and line, that led to an error. The top line is where to look first.

status code

The number that tells the client how it went: 200 OK, 201 Created, 404 Not Found, 500 Server Error.

throws

Raise an error. Only Error objects can be thrown.

timestamp

A date and time value. A column like last_update can fill itself with the current time.

URL

The address of a resource: protocol, host, port and path, such as http://localhost:3000/user.